

Southwest
Tech

Web Development Foundations

Week 6 - Debugging: "Things Break - And I Can Fix Them"
Day 1: "Debugging Process"

1

Southwest
Tech

Debugging is INFORMATION.



Debugging is **not** proof that you failed.

Debugging means:

- ▶ something happened
- ▶ we can observe it
- ▶ the browser may give clues
- ▶ one focused change can be tested

Problems are information.

2

Southwest
Tech

Debugging: Calm Information Gathering

We observe, collect clues, make a small change, and verify.



Problems are information.

Debugging As Information

3

Lab Review: Successful Path

One successful path:

- ▶ JavaScript linked separately
- ▶ page elements selected
- ▶ click event connected
- ▶ input value read
- ▶ visible feedback updated
- ▶ no console errors

Next, we practice recovery when that chain breaks.

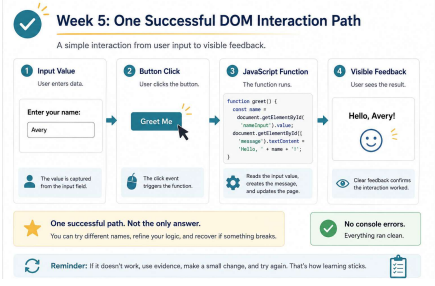


4

Southwest
Tech

Week 5: One Successful DOM Interaction Path

A simple interaction from user input to visible feedback.



One successful path. Not the only answer.
You can try different names, refine your logic, and recover if something breaks.

No console errors.
Everything ran clean.

Reminder: If it doesn't work, use evidence, make a small change, and try again. That's how learning sticks.

Week 5 Success Path

5

Southwest
Tech

From: Working Interaction To: Broken Connection

A broken interaction may look simple:

- ▶ button does nothing
- ▶ message does not change
- ▶ input is ignored
- ▶ layout looks wrong
- ▶ console shows an error

Do not guess first. -- Observe first.

6

Southwest Tech

A Previously Working DOM Connection with One Broken Link

The page used to respond. Now one link is broken.

1. HTML Element
The element exists on the page.

2. Selector
Looks for the element using a selector.

3. Event
User clicks the button.

4. Function
The function runs when the event fires.

5. Visible Update
User sees the result on the page.

Find the broken connection before changing everything.
Check the selector carefully. If it does not match the HTML, the chain cannot start.

Good Debug Habit
Change one thing at a time. Repeat and confirm.

Once repaired: HTML element → selector → event → function → visible update. The page responds again.

Broken Connection

7

Southwest Tech

What Counts as Success Today

- you reproduce the problem
- you read one clue
- you compare the related code
- you make one focused change
- you retest and explain the result

8

Southwest Tech

What We Will Use Today

- reproduce
- observe
- console
- error message
- HTML id
- JavaScript selector
- one focused change
- retest

9

Southwest Tech

Today's Beginner Debugging Toolbox

Simple tools. Clear steps. Small fixes.

1. Reproduce
Make it happen again.

2. Observe
What do you see?

3. Console
Check what the browser reports.

4. Error Message
Read the message carefully.

5. HTML ID
Check for unique IDs.

6. ID Selector
Check the selector matches the ID.

7. One Focused Change
Change one thing at a time.

8. Retest
Does it work now?

Use evidence. Make one change. Retest.

Debugging Toolbox

10

Southwest Tech

What We Will "Skip" For Now

- advanced breakpoints
- full stack traces
- performance tools
- browser compatibility issues
- automated tests
- large debugging sessions

Today:
one bug,
one clue,
one fix.

11

Southwest Tech

Advanced Debugging Topics Parked for Later

These topics are important. They just aren't the focus right now. We'll return to them when you're ready.

Advanced Breakpoints
Conditional, logical, and more.

Full Stack Traces
Reading deep call stacks and errors.

Performance Tools
Profiling, timelines, and memory views.

Browser Compatibility
Cross-browser testing and quirks.

Automated Tests
Live, integration and end-to-end testing.

Large Debugging Sessions
Strategies for complex, multi-hour bugs.

Not today.
First, one bug, one clue, one fix.

Later:
Gather one clue.
Make one focused fix.
Confirm and move on.

Parked For Later

12

Debugging Is A Loop

Use the debugging loop:

1. reproduce the problem
2. observe the evidence
3. isolate one likely cause
4. change one thing
5. retest
6. explain what happened

13

Beginner Debugging Loop

A simple loop you can repeat until it works.

Debugging is a process, not a panic response.

- Stay calm. Small steps create clarity.
- You will get better with practice.
- Every bug you fix builds your skill.

Debugging Loop

14

Symptom Is Not Cause



Symptom:
"The button does nothing."

Possible causes:

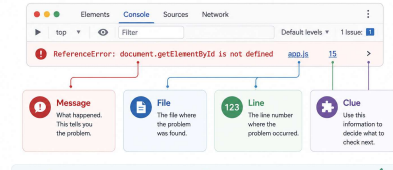
- ▶ script file is not linked
- ▶ selector does not match
- ▶ function is not connected
- ▶ code has an earlier error

Name the symptom ... *Then* gather evidence for the cause.

15

Browser Console Message as a Clue

The console is giving you information—use it.



Message What happened. This tells you the problem.

File The file where the problem was found.

Line The line number where the problem occurred.

Clue Use this information to decide what to check next.

Read the message before changing code.

- ✓ The console is trying to help you.
- ✓ One good clue can save you time.
- ✓ Small steps lead to clear fixes.

Symptom vs. Cause

16

Console Messages Are Clues



The console may tell you:

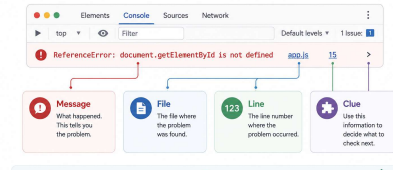
- ▶ which file had a problem
- ▶ which line was involved
- ▶ what kind of problem happened
- ▶ what value was missing or unexpected

Read the message before changing code.

17

Browser Console Message as a Clue

The console is giving you information—use it.



Message What happened. This tells you the problem.

File The file where the problem was found.

Line The line number where the problem occurred.

Clue Use this information to decide what to check next.

Read the message before changing code.

- ✓ The console is trying to help you.
- ✓ One good clue can save you time.
- ✓ Small steps lead to clear fixes.

Console Clue

18



AI Can Explain Clues, Not Replace Evidence

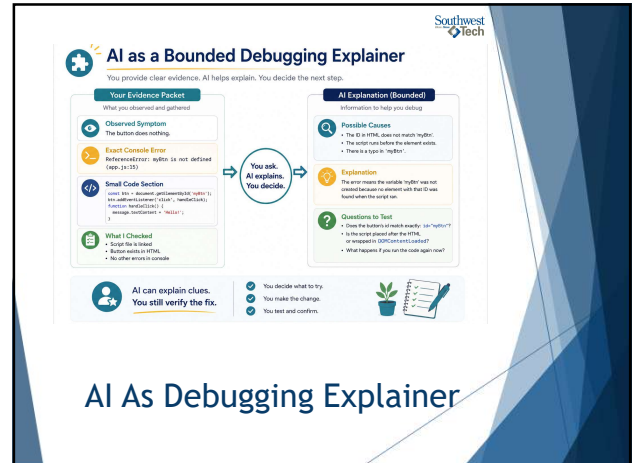
AI can help explain debugging clues.

Useful AI input:

- what you observed
- the exact console error
- the small code section involved
- what you already checked

AI should help you understand possible causes.
It should **not** replace your evidence or verification.

19



AI as a Bounded Debugging Explainer

You provide clear evidence. AI helps explain. You decide the next steps.

Your Evidence Packet

- What you observed and gathered
- Observed Symptom: The button does nothing.
- Exact Console Error: ReferenceError: btn is not defined (app.js:15)
- Small Code Section:

```
const btn = document.getElementById('btn');
btn.addEventListener('click', () => {
  console.log('Clicked!');
});
```
- What I Checked:
 - Single fix is tested.
 - Button exists in HTML.
 - No other errors in console.

AI Explanation (Bounded)

Information to help you debug:

- Possible Causes:**
 - The ID in HTML does not match what the script expects.
 - The script is before the element exists.
 - There is a typo in 'btn'.
- Explanation:** The error means the variable 'btn' was not assigned before or doesn't exist with that ID when found when the script ran.
- Questions to Test:**
 - Check the ID in HTML: is it 'btn' or 'statusButton'?
 - Is the script placed after the HTML?
 - Are there typos in the code?
 - What happens if you run the code again?

AI can explain clues. You still verify the fix.

- You decide what to try.
- You make the changes.
- You test and confirm.

20



Useful AI Prompt Pattern

I am debugging a beginner web page.

What I observed:
[What I clicked or tried, and what happened]

Console message:
[Paste the exact message]

Relevant code:
[Paste the small HTML/JS section]

What I checked:
[IDs, selectors, script link, spelling]

Please explain what this error means.
Give me possible causes to test.
Do not rewrite the whole project.

21



Compare What Must Match

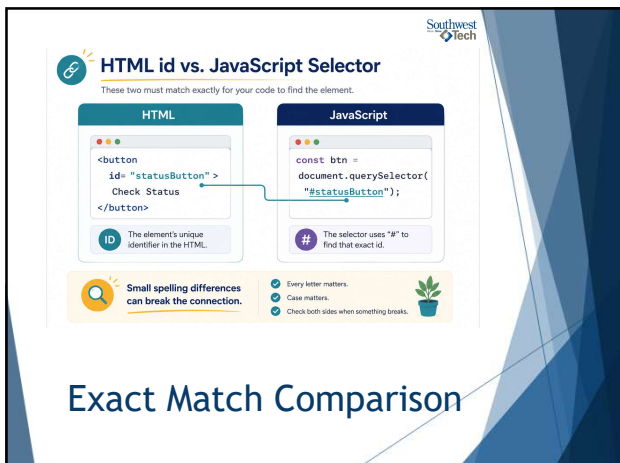
Some code must match exactly.

HTML:
``id="statusButton"``

JavaScript:
``document.querySelector("#statusButton")``

Small spelling differences can break the connection.

22



HTML id vs. JavaScript Selector

These two must match exactly for your code to find the element.

HTML:

```
<button id="statusButton">
  Check Status
</button>
```

The element's unique identifier in the HTML: **ID**

JavaScript:

```
const btn =
  document.querySelector(
    "#statusButton");
```

The selector uses **#** to find that exact id.

Small spelling differences can break the connection.

- Every letter matters.
- Case matters.
- Check both sides when something breaks.

Exact Match Comparison

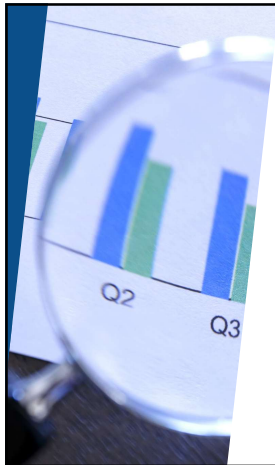
23



Demo

"Broken Selector"

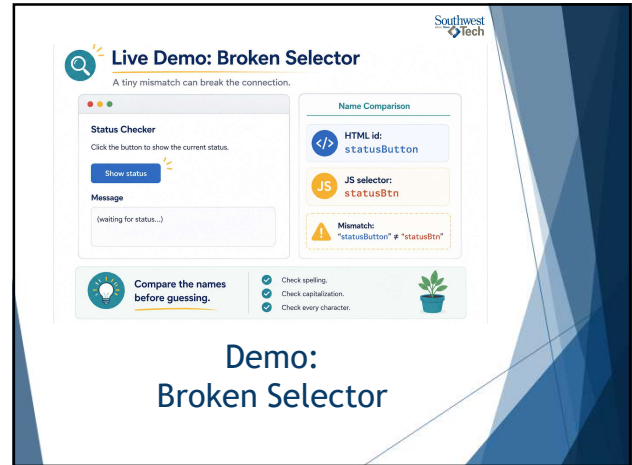
24



What to Watch For

- ▶ reproduce the problem
- ▶ open the console
- ▶ read the error
- ▶ compare HTML id to JS selector
- ▶ fix one selector
- ▶ refresh and retest

25



Demo: Broken Selector

26



Verify The Fix

After a fix:

- ▶ save the file
- ▶ refresh the page
- ▶ repeat the original action
- ▶ check the console again
- ▶ explain what changed

Do not stop at "I changed something."
Stop at "I verified the result."

27



Verify The Fix

28



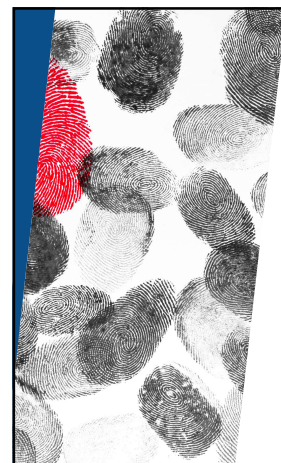
Lab

First debugging pass

Your goal:

- ▶ identify at least 3 issues
- ▶ describe what is wrong
- ▶ attempt focused fixes
- ▶ test each change
- ▶ keep notes on what you observed

29



Evidence

Preserve evidence:

- ▶ issue observed
- ▶ clue or evidence found
- ▶ file or line inspected
- ▶ change attempted
- ▶ result after retesting

Clear notes are part of the work.

30

A photograph of laboratory glassware, including a beaker and a graduated cylinder, both containing a blue liquid. The background is a blurred laboratory setting.

Southwest
Tech

Closing Success Check

Today:

- ▶ broken behavior became evidence
- ▶ console messages became clues
- ▶ matching code mattered
- ▶ one focused fix was verified

Next:

- ▶ we debug more than one issue without chasing everything at once.

31

A large blue thumbs-up icon.

Southwest
Tech

Thank you! Have Fun!

32